PROJECT REPORT

Devixo Solutions

AI/ML Internship Program

Task 04 – End-to-End ML Application with Deployment

Intern Name: Shafaque Shahid
Internship Domain: AI/ML
Date: 04/11/2026

1. Introduction

The Movie Preference Prediction and Recommendation System is an end-to-end machine learning project developed as part of the Devixo Solutions Machine Learning Internship. The system analyzes historical movie ratings to predict whether a user is likely to enjoy a movie. It then recommends movies that the selected user has not previously rated. The trained machine learning model is integrated into an interactive Streamlit application.

2. Project Objectives

- Understand and preprocess a real-world movie ratings dataset.
- Perform feature engineering and prepare data for machine learning.
- Train and compare at least four classification algorithms.
- Apply hyperparameter tuning to improve model performance.
- Evaluate the final model using classification metrics.
- Save the trained model using Joblib.
- Develop an interactive application using Streamlit.

3. Dataset Description

The project uses the MovieLens Latest-Small dataset, which contains movie information and historical ratings provided by users.

Dataset Information	Count
Movies	9,742
Ratings	100,836
Unique Users	610

The ratings were converted into binary preference labels. Ratings of 4.0 or higher were classified as Liked (1), while ratings below 4.0 were classified as Not Liked (0). Movie genres were encoded into binary features.

4. Data Preprocessing and Feature Engineering

1. Loaded the movies.csv and ratings.csv files.
2. Prepared a binary target variable from numerical ratings.
3. Encoded movie genres using one-hot encoding.
4. Split the data into 80% training and 20% testing sets using stratification.
5. Generated user statistics, including average rating, rating count, and like rate.
6. Generated movie statistics, including average rating, rating count, and like rate.
7. Used out-of-fold statistics during training feature generation to reduce target leakage.

5. Machine Learning Models

Four classification algorithms were trained and evaluated to identify the most suitable model for predicting movie preferences.

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Random Forest
- Gradient Boosting

Gradient Boosting was selected for further optimization because it provided the strongest overall results among the initial models.

6. Hyperparameter Tuning

RandomizedSearchCV was applied to the Gradient Boosting classifier using cross-validation and F1-score as the optimization metric. The tuned model was evaluated on the held-out test set.

7. Final Model Performance

The tuned Gradient Boosting classifier achieved the following evaluation results on the test dataset:

Metric	Score
Accuracy	71.57%
Precision	69.52%
Recall	72.98%
F1-score	71.21%
ROC-AUC	79.01%

Hyperparameter tuning resulted in a small improvement over the original Gradient Boosting model. The F1-score increased from 0.7105 to 0.7121, while ROC-AUC increased from 0.7891 to 0.7901.

8. Model Saving

The final tuned Gradient Boosting pipeline was saved using Joblib in a pickle file. The Streamlit application loads this saved model to generate predictions without retraining the model every time the application starts.

9. Streamlit Application

An interactive web application was developed using Streamlit. It provides the following sections:

Home: Displays the project introduction and dataset statistics.

Movie Recommendations: Allows users to select an existing MovieLens User ID, choose the number of recommendations, filter by genre, and view predicted preference scores.

Movie Search: Allows users to search movies by title and view release year and genres.

Dataset Overview: Displays movie and rating samples and a rating distribution chart.

Model Performance: Displays evaluation metrics and feature importance.

10. How to Run the Application

The following files must be placed in the same project directory:

- app.py
- movie_recommendation_model (1).pkl
- movies.csv
- ratings.csv
- requirements.txt

Install the required libraries:

pip install -r requirements.txt

Start the Streamlit application:

streamlit run app.py

The application can then be accessed through the local URL displayed by Streamlit. The model filename used in app.py must match the actual model file.

11. Limitations

- The current recommendation system supports existing MovieLens users with historical ratings.
- New users without rating history cannot yet receive fully personalized recommendations.
- Prediction scores are model scores and are not guaranteed probabilities of user satisfaction.
- The application uses a relatively small public dataset and is an internship prototype.
- A successful local application run does not by itself establish permanent public deployment.

12. Future Improvements

- Add a new-user onboarding feature using selected genres or initial movie ratings.
- Integrate movie posters and additional movie information.
- Add user accounts and saved preferences.
- Evaluate recommendations using Precision@K, Recall@K, and NDCG.
- Deploy the application to a public cloud hosting platform.

13. Conclusion

This project demonstrates an end-to-end machine learning workflow, including data preprocessing, feature engineering, model training, model comparison, hyperparameter tuning, evaluation, model saving, and application development. The final tuned Gradient Boosting model is integrated into a Streamlit application that provides personalized movie recommendations for existing MovieLens users, movie search, dataset exploration, and model performance information. The result is an interactive machine learning application rather than only a Jupyter Notebook.